

SIRA

Sistema de Reserva de Salas - IFPB

Documento Consolidado

Épicos · Features · Tarefas dos Membros

Reúne o backlog de produto, o mapeamento das responsabilidades e o conteúdo integral dos PDFs individuais entregues pelos 5 membros da equipe.

Projeto	SIRA v1.0.0 - Sistema de Reserva de Salas e Equipamentos
Stack	Vite 8 + Vanilla JS (ESM) + LocalStorage
Disciplinas	Programação para Web 2 + Engenharia de Requisitos de Software
Instituição	IFPB - Instituto Federal da Paraíba
Stakeholders	Coord. Diego Pessoa e Coord. Paulo Ditarso (entrevistas)
Equipe	5 desenvolvedores (Gabriel, Ian, Igor, José, Pedro)
Data	2026-05-07

Sumário

#	Seção
1	Visão geral do produto
2	Personas, perfis e restrições técnicas
3	Status atual da entrega
4	Mapa de Features (Epic → Feature → User Story)
5	Mapeamento: Épicas -> Features -> Membros
6	Catálogo completo de Épicas e User Stories
7	Resumo quantitativo
8	Tarefas individuais - Gabriel Marques
9	Tarefas individuais - Ian Lucas
10	Tarefas individuais - Igor Gimenez
11	Tarefas individuais - José Henrique
12	Tarefas individuais - Pedro Sales
13	Workflow Git e checklist final

1. Visão geral do produto

SIRA é uma SPA construída em **JavaScript puro (Vanilla ESM)** sobre o **Vite**, sem framework de UI. O sistema centraliza a reserva de salas, laboratórios e auditórios do IFPB, endereço direto das dores levantadas em entrevistas com os coordenadores Diego Pessoa e Paulo Ditarso - principalmente a falta de sincronização entre o SUAP e o HIFPB.

O que o sistema faz

- Permite que **professores** consultem o calendário semanal de salas, criem solicitações de reserva (com recorrência semanal opcional) e acompanhem o status das mesmas.
- Permite que o **administrador** aprove/recuse solicitações em fluxo de 1º e 2º nível, gerencie o cadastro de salas e usuários, monitore notificações e revise a integração SUAP/HIFPB.
- Persiste todos os dados em **LocalStorage** com partição por usuário (sira_db/<email>/<coleção>.json), simulando um banco real e permitindo demonstração offline.

Stack técnica

Camada	Tecnologia
Linguagem	JavaScript (ESM) + HTML5 + CSS3
Build / Dev	Vite ^8.0.10
Linter / Formatter	ESLint 9 + Prettier 3
Hooks	Husky 9 + lint-staged 15
Persistência	localStorage (sem backend)
Roteamento	History API (sem biblioteca)
CI/CD	GitHub Actions -> GitHub Pages (release-driven)
Node alvo	Node.js 24 LTS

2. Personas, perfis e restrições técnicas

Persona	Role	Responsabilidades
Professor / Coordenador	professor	Solicita e gerencia suas próprias reservas, consulta o calendário semanal
Administrador	admin	Aprova reservas, gerencia salas, usuários, cadastros e notificações
Sistema externo	(externo)	SUAP / HIFPB - escopo futuro declarado mas ainda não implementado

Restrições pedagógicas (critérios das disciplinas)

- Programação funcional explícita: `map` / `filter` / `reduce`.
- Módulos ESM nativos (`"type": "module"`).
- Estruturação de dados em arrays + persistência via JSON em `LocalStorage`.
- Tratamento de eventos (`input`, `click`, `keydown`, `popstate`).
- Manipulação do DOM via `createElement` / `appendChild`, encapsulada em `el()`.
- Vite com template Vanilla (sem React/Vue/etc.).

3. Status atual da entrega

Snapshot do repositório em **2026-05-11**. As marcações de done/pendente no restante do documento refletem este estado real - não a previsão otimista do plano original.

Branch	Estado	Cobertura
main	Estável	Recebe releases de develop; última release pós-sprint 1 em preparação
develop	Integração contínua	15 user stories mergeadas (US-01..15 + US-25); 15 features ativas (F-01..F-14 + F-24)
feature/* (5 ramos abertos)	PRs pendentes	10 user stories restantes (US-16..24); 9 features (F-15..F-23) cobrindo CRUD de reservas, admin e dashboard

Features já consolidadas em develop

- **F-01** Scaffold Vite + ESM + GitHub Pages · US-01 · Gabriel · PR #125
- **F-02** Biblioteca de utilitários e design system base · US-02 · Gabriel · PR #126
- **F-03** Login institucional com sessão persistente · US-03 · Gabriel · PR #127
- **F-04** Auto-serviço de cadastro de professor · US-04 · Gabriel · PR #128
- **F-05** Encerramento de sessão · US-05 · Gabriel + Ian · PRs #127 (logout) e #134 (botão Sair)
- **F-06** Navegação contextual por perfil (sidebar + URLs limpas) · US-06, US-07 · Ian · PRs #134 e #144
- **F-07** Adaptação responsiva mobile (drawer + tabelas em cards) · US-08 · Ian · PR #145
- **F-08** Tema claro/escuro persistente · US-09 · Ian + Gabriel · PRs #134 (toggle) e #154 (restore no bootstrap)
- **F-09** Dados isolados por usuário · US-10 · Igor · PR #143
- **F-10** Sincronização aprovação → reserva → notificação · US-11 · Igor · PR #143
- **F-12** Grade semanal de reservas · US-13 · Ian · PR #147
- **F-13** Busca anti-conflito de salas · US-14 · José · PR #152
- **F-14** Reserva express com 1 clique · US-15 · José · PR #152
- **F-24** Sistema de modais centralizado · US-25 · Ian · PR #149

Features pendentes (escopo restante)

- **F-11** Painel administrativo de KPIs em tempo real · US-12 · Igor · computeStats já existe em fp.js, falta consumi-la em dashboard.js.
- **F-15** Listagem de reservas pessoais com filtros e busca · US-16 · José · src/modules/reservations.js.
- **F-16** Edição e cancelamento de reservas pendentes · US-17 · José · modais 'Ver detalhes' e 'Editar' + cancelamento com confirmação.
- **F-17** Exportação de reservas para CSV · US-18 · José · exportCSV() via Blob + URL.createObjectURL.
- **F-18** Fila consolidada de aprovações pendentes · US-19 · Pedro · src/modules/approvals.js.
- **F-19** Decisão de aprovação com notificação ao autor · US-20 · Pedro · resolveApproval + atualização de badge.
- **F-20** CRUD de salas com filtros e recursos · US-21 · Pedro · src/modules/rooms.js.

- **F-21** CRUD de usuários · US-22 · Pedro · `src/modules/users.js`.
- **F-22** Revisão de solicitações de cadastro pendentes · US-23 · Pedro · `modal Solicitações de Cadastro` consumindo `localStorage["sira:signups"]`.
- **F-23** Caixa de notificações com marcação como lida · US-24 · Igor · `src/modules/notifications.js`.

Ordem de implementação recomendada para o que falta

#	Membro	Bloco a entregar (Features)	Por que vem nesta ordem
1	José Henrique	F-15, F-16, F-17 (US-16, 17, 18) — CRUD de reservas pessoais + CSV	Já entregou F-13 e F-14 (Nova Reserva); fluxo natural é o usuário ver, editar e exportar as próprias reservas que acabou de criar.
2	Pedro Sales	F-18, F-19 (US-19, 20) — fila + decisão de aprovação	Resolve aprovações geradas pelo fluxo de criação do José (Nova Reserva produz uma aprovação pendente).
3	Pedro Sales (cont.)	F-20 (US-21) — CRUD de salas	Inventário físico é necessário para Nova Reserva listar opções reais; pode rodar em paralelo com F-18/19.
4	Pedro Sales (cont.)	F-21, F-22 (US-22, 23) — CRUD e revisão de cadastros de usuários	Consome dados de US-04 (signups) que já estão sendo gravados; viabiliza onboarding ponta-a-ponta.
5	Igor Gimenez	F-11, F-23 (US-12, 24) — Dashboard + Notificações	São consumidores finais: agregam reservas/aprovações criadas pelas features anteriores.

4. Mapa de Features (Epic → Feature → User Story)

Convenção de níveis usada neste documento:

- **Épico (EP-XX)**: grande iniciativa de produto, geralmente cobrindo um domínio funcional. Define o *porquê* macro.
- **Feature (F-XX)**: capacidade entregue ao usuário, agrupa user stories relacionadas que entregam um conjunto coeso de valor. Define o *o quê* (observável de fora).
- **User Story (US-XX)**: incremento implementável do ponto de vista do ator. Define o *como* (granularidade de PR/commit).

O SIRA tem **13 épicos** que se decompõem em **24 features** e **25 user stories**. A tabela abaixo é o mapa completo, ordenado por épico. Cada feature é nomeada pelo valor que entrega — não pelo arquivo técnico que a implementa — para preservar a comunicação com stakeholders.

Épico	Feature	User stories
EP-01 · Fundação Técnica do Projeto	F-01 · Scaffold Vite + ESM com deploy para GitHub Pages	US-01
	F-02 · Biblioteca de utilitários funcionais e design system base	US-02
EP-02 · Autenticação e Sessão	F-03 · Login institucional com sessão persistente	US-03
	F-04 · Auto-serviço de cadastro de professor	US-04
	F-05 · Encerramento de sessão	US-05
EP-03 · Shell, Navegação e Roteamento	F-06 · Navegação contextual por perfil (sidebar + URLs limpas)	US-06, US-07
	F-07 · Adaptação responsiva mobile (drawer + tabelas em cards)	US-08
	F-08 · Tema claro/escuro persistente	US-09
EP-04 · Camada de Persistência	F-09 · Dados isolados por usuário no LocalStorage	US-10
	F-10 · Sincronização aprovação → reserva → notificação	US-11
EP-05 · Dashboard (Admin)	F-11 · Painel administrativo de KPIs em tempo real	US-12
EP-06 · Calendário (Home)	F-12 · Grade semanal de reservas (7d × 12h)	US-13
EP-07 · Nova Reserva	F-13 · Busca anti-conflito de salas	US-14
	F-14 · Reserva express com 1 clique	US-15
EP-08 · Minhas Reservas (CRUD)	F-15 · Listagem de reservas pessoais com filtros e busca	US-16
	F-16 · Edição e cancelamento de reservas pendentes	US-17
	F-17 · Exportação de reservas para CSV	US-18

Épico	Feature	User stories
EP-09 · Fluxo de Aprovações	F-18 · Fila consolidada de aprovações pendentes	US-19
	F-19 · Decisão de aprovação com notificação ao autor	US-20
EP-10 · Gestão de Salas (Admin)	F-20 · CRUD de salas com filtros e recursos	US-21
EP-11 · Gestão de Usuários (Admin)	F-21 · CRUD de usuários	US-22
	F-22 · Revisão de solicitações de cadastro pendentes	US-23
EP-12 · Notificações	F-23 · Caixa de notificações com marcação como lida	US-24
EP-13 · Componentes Reutilizáveis	F-24 · Sistema de modais centralizado	US-25

5. Mapeamento: Épicos → Features → Membros

A tabela abaixo cruza épicos, features e os membros responsáveis. Cada linha representa um épico do produto, com os IDs F-XX das features que o compõem e as user stories cobertas. O detalhamento de cada feature está na seção anterior; o detalhamento de cada user story está na próxima seção.

Épico	Features (F-XX)	User stories	Responsável	Status
EP-01 Fundação Técnica do Projeto	F-01, F-02	US-01, US-02	Gabriel Marques	Concluído
EP-02 Autenticação e Sessão	F-03, F-04, F-05	US-03, US-04, US-05	Gabriel Marques (F-03/04/05.T-05.2) + Ian Lucas (F-05.T-05.1)	Concluído
EP-03 Shell, Navegação e Roteamento	F-06, F-07, F-08	US-06, US-07, US-08, US-09	Ian Lucas (F-06/07) + Gabriel Marques (F-08)	Concluído
EP-04 Camada de Persistência	F-09, F-10	US-10, US-11	Igor Gimenez	Concluído
EP-05 Dashboard (Admin)	F-11	US-12	Igor Gimenez	Pendente
EP-06 Calendário (Home)	F-12	US-13	Ian Lucas	Concluído
EP-07 Nova Reserva	F-13, F-14	US-14, US-15	José Henrique	Concluído
EP-08 Minhas Reservas (CRUD)	F-15, F-16, F-17	US-16, US-17, US-18	José Henrique	Pendente
EP-09 Fluxo de Aprovações	F-18, F-19	US-19, US-20	Pedro Sales	Pendente
EP-10 Gestão de Salas (Admin)	F-20	US-21	Pedro Sales	Pendente
EP-11 Gestão de Usuários (Admin)	F-21, F-22	US-22, US-23	Pedro Sales	Pendente
EP-12 Notificações	F-23	US-24	Igor Gimenez	Pendente
EP-13 Componentes Reutilizáveis	F-24	US-25	Ian Lucas	Concluído

Distribuição por membro

Membro	Tema	Features	User Stories	Done / Total
Gabriel Marques	Fundação + Autenticação + Tema	F-01, F-02, F-03, F-04, F-05, F-08	US-01, 02, 03, 04, 05, 09	6 / 6
Ian Lucas	Shell + Navegação + Modais + Calendário	F-05, F-06, F-07, F-12, F-24	US-05.T-05.1, 06, 07, 08, 13, 25	5 / 5

Membro	Tema	Features	User Stories	Done / Total
Igor Gimenez	Persistência + Dashboard + Notificações	F-09, F-10, F-11, F-23	US-10, 11, 12, 24	2 / 4
José Henrique	Reservas (Nova Reserva + CRUD + CSV)	F-13, F-14, F-15, F-16, F-17	US-14, 15, 16, 17, 18	2 / 5
Pedro Sales	Admin (Aprovações + Salas + Usuários)	F-18, F-19, F-20, F-21, F-22	US-19, 20, 21, 22, 23	0 / 5

6. Catálogo completo de Épicas e User Stories

Épico 1 - Fundação Técnica do Projeto

US-01 · Como dev da equipe, quero um setup mínimo Vite vanilla rodando, para que possa iterar rapido sem framework

Feature: F-01 · Scaffold Vite + ESM com deploy para GitHub Pages

Critérios de aceitação

- `npm install && npm run dev` sobe servidor em `http://localhost:5173`
- `npm run build` gera artefatos em `dist/`
- Projeto usa ESM nativo (`"type": "module"`)

Tasks técnicas

- [x] T-01.1 - Inicializar package.json com scripts dev/build/preview e Vite ^5.2.0
- [x] T-01.2 - Criar vite.config.js com root: '.' e outDir: 'dist'
- [x] T-01.3 - Criar index.html com `<script type="module" src="/src/main.js">`
- [x] T-01.4 - Configurar .gitignore para node_modules/ e dist/
- [x] T-01.5 - Documentar setup no README.md

US-02 · Como dev, quero utilitários funcionais reutilizáveis, para que a manipulação de listas e DOM siga um padrão único

Feature: F-02 · Biblioteca de utilitários funcionais e design system base

Tasks técnicas

- [x] T-02.1 - src/utils/fp.js: filterByText, filterByStatus, filterRoomsByStatus, computeStats (reduce), initials, roleLabel, statusBadge, roomStatusInfo
- [x] T-02.2 - src/utils/dom.js: el(), render(), btn(), badge(), tableRow(), toast(), confirm()
- [x] T-02.3 - CSS global (style.css, home.css, auth.css) com CSS Variables e dark mode

Épico 2 - Autenticação e Sessão

US-03 · Como usuário, quero entrar no sistema com meu e-mail institucional, para que minhas reservas fiquem associadas a mim

Feature: F-03 · Login institucional com sessão persistente

Critérios de aceitação

- Tela de login renderizada quando `CURRENT_USER === null`
- Login válido persiste sessão em `localStorage["sira-auth"]`
- Login inválido exibe alerta "Usuário não encontrado"

Tasks técnicas

- **[x] T-03.1** - Implementar `login()`, `logout()`, `tryRestoreSession()` em `src/data/store.js`
- **[x] T-03.2** - Criar tela de login inline em `src/main.js`
- **[x] T-03.3** - Bootstrapping com `tryRestoreSession()` antes de montar shell
- **[x] T-03.4** - Seed do admin em `src/data/logins.json`

US-04 · Como visitante, quero solicitar cadastro como professor, para que o admin aprove minha entrada no sistema

Feature: F-04 · Auto-serviço de cadastro de professor

Critérios de aceitação

- Formulário com nome, e-mail, perfil (professor)
- Solicitação persiste em `localStorage["sira:signups"]` com `approved: false`
- Após envio, mostra alerta de confirmação e volta ao login

Tasks técnicas

- **[x] T-04.1** - Implementar `renderSignup()` em `src/main.js`
- **[x] T-04.2** - Validação de campos obrigatórios (nome + e-mail)
- **[x] T-04.3** - Geração de ID `su-<timestamp>` para a solicitação

US-05 · Como usuário logado, quero sair do sistema, para que minha sessão não fique aberta

Feature: F-05 · Encerramento de sessão

Tasks técnicas

- **[] T-05.1** - Botão "Sair" no `userPill` da sidebar em `src/components/sidebar.js`
- **[] T-05.2** - Limpeza de `localStorage["sira-auth"]` ao sair

Depende de US-06 - `logout` fica dentro do `userPill` da sidebar, que ainda não foi criada. `logout()` já existe em `store.js`, falta o ponto de entrada na UI.

Épico 3 - Shell, Navegação e Roteamento

US-06 · Como usuário, quero uma navegação lateral com seções por contexto, para que possa transitar entre páginas com 1 clique

Feature: F-06 · Navegação contextual por perfil (sidebar + URLs limpas)

Critérios de aceitação

- Sidebar com seções VISAO GERAL, RESERVAS, ADMINISTRACAO
- Itens visíveis variam por role (admin vs. professor)
- Item ativo destacado e badges numéricos para reservas/aprovações/notificações não lidas

Tasks técnicas

- **[] T-06.1** - Componente src/components/sidebar.js com NAV_ITEMS declarativo
- **[] T-06.2** - Filtro por role: professor ve apenas reservas, calendário, novaReserva
- **[] T-06.3** - Badges dinamicos via funções badge: () => ... lendo store
- **[] T-06.4** - window.updateSidebarBadges para atualização imperativa após mutações

US-07 · Como usuário, quero URLs limpas e suporte a voltar/avancar, para compartilhar links e usar o histórico do navegador

Feature: F-06 · Navegação contextual por perfil (sidebar + URLs limpas)

Tasks técnicas

- **[] T-07.1** - Roteador funcional PAGE_RENDERERS em src/main.js
- **[] T-07.2** - window.history.pushState em navigate()
- **[] T-07.3** - Listener de popstate em src/main.js
- **[] T-07.4** - Fallback para calendário quando rota desconhecida ou usuário não-admin tenta acessar página restrita

US-08 · Como usuário mobile, quero abrir/fechar a sidebar como drawer, para que a interface caiba em telas pequenas

Feature: F-07 · Adaptação responsiva mobile (drawer + tabelas em cards)

Tasks técnicas

- **[] T-08.1** - Hamburger injetado dinamicamente via MutationObserver em src/main.js
- **[] T-08.2** - Overlay clicavel que fecha o drawer
- **[] T-08.3** - addTableLabels() injeta data-label para tabelas responsivas

US-09 · Como usuário, quero alternar tema claro/escuro, para que possa trabalhar em qualquer iluminação

Feature: F-08 · Tema claro/escuro persistente

Tasks técnicas

- **[] T-09.1** - Toggle dark-toggle no rodapé da sidebar
- **[] T-09.2** - Persistência em localStorage["sira-theme"]
- **[] T-09.3** - Aplicação na <html>.dark no bootstrap

Épico 4 - Camada de Persistência (LocalStorage por Usuário)

US-10 · Como sistema, preciso isolar dados por usuário logado, para que reservas de um professor não vazem para outro

Feature: F-09 · Dados isolados por usuário no LocalStorage

Critérios de aceitação

- Caminho de persistência: `sira_db/<email>/<coleção>.json`
- Sem usuário logado, `loadCollection` retorna apenas o seed
- Admin consegue consolidar dados de todos os usuários (reservas, aprovações)

Tasks técnicas

- **[] T-10.1** - `loadCollection/saveCollection` por path em `src/data/store.js`
- **[] T-10.2** - Getters: `getRooms`, `getReservations`, `getUsers`, `getNotifications`, `getApprovals`
- **[] T-10.3** - Consolidação admin em `getReservations` e `getApprovals`
- **[] T-10.4** - `saveRooms/saveUsers` propagando para todos os usuários quando admin salva
- **[] T-10.5** - `genId(prefix)` para chaves únicas
- **[] T-10.6** - Seeds vazios em `src/data/seed.json`

Pré-requisito de TODA a camada de produto (US-12 a US-24). Começar por aqui.

US-11 · Como admin, quero resolver uma aprovação e ver isso refletido nas reservas e notificações do solicitante, para que o fluxo fique consistente

Feature: F-10 · Sincronização aprovação → reserva → notificação

Tasks técnicas

- **[] T-11.1** - `resolveUserApproval(approvalId, email, decision)` em `src/data/store.js`
- **[] T-11.2** - Remove aprovação, atualiza status da reserva e cria notificação no diretório do usuário-alvo

Recomendação de implementação - vincular aprovação a reserva por `reservationId` determinístico (gerado em `performReservation`), não por heurística de string ou `date+time`.

Épico 5 - Dashboard (Admin)

US-12 · Como admin, quero ver estatísticas em tempo real (total de salas, disponíveis, reservas pendentes, ocupação), para que tenha visão geral instantânea

Feature: F-11 · Painel administrativo de KPIs em tempo real

Tasks técnicas

- **[x] T-12.1** - `computeStats` com `Array.reduce` em `src/utils/fp.js`
- **[] T-12.2** - Render de 4 stat-cards via `Array.map` em `src/modules/dashboard.js`
- **[] T-12.3** - Tabela "Reservas aprovadas recentes" (slice 3)
- **[] T-12.4** - Placeholder de gráfico de ocupação ("integração futura HIFPB")

`computeStats` já existe em `fp.js` (US-02). Faltava o módulo `dashboard.js` consumi-la.

Épico 6 - Calendário (Home)

US-13 · Como usuário, quero ver as reservas da semana em uma grade dia x hora, para que eu enxergue conflitos visualmente

Feature: F-12 · Grade semanal de reservas (7d × 12h)

Critérios de aceitação

- Grade 7 dias x 12 horas (07:00 - 18:00)
- Eventos coloridos por status (approved=verde, pending=âmbar, rejected=rosa)
- Navegação semana anterior / próxima

Tasks técnicas

- **[] T-13.1** - getWeekDates(offset) em src/modules/calendar.js
- **[] T-13.2** - reservationsToEvents (parsing data/hora + spread em blocos)
- **[] T-13.3** - Render da grade dinamica (rebuildCalendar)

Épico 7 - Nova Reserva (Busca + Criação com Anti-Conflito)

US-14 · Como professor, quero buscar salas disponíveis informando data, horário, tipo e finalidade, para encontrar opções compatíveis sem precisar olhar tabela

Feature: F-13 · Busca anti-conflito de salas

Critérios de aceitação

- Filtro por tipo (Sala/Laboratório/Auditório)
- Filtro por sobreposição de horário (exclui salas com reserva no intervalo solicitado, exceto rejected)
- Validação timeStart < timeEnd
- Suporte a recorrência semanal (UI presente, lista de dias da semana selecionaveis)

Tasks técnicas

- **[] T-14.1** - Formulario em src/modules/novaReserva.js
- **[] T-14.2** - parseTimeStr para tempos com 'h' ou ':'
- **[] T-14.3** - searchRooms com filtro de overlap
- **[] T-14.4** - Botões dia-da-semana com toggle visual

US-15 · Como professor, quero ver detalhes da sala e reservar com 1 clique, para que o fluxo seja rapido

Feature: F-14 · Reserva express com 1 clique

Tasks técnicas

- **[] T-15.1** - showRoomDetailsModal
- **[] T-15.2** - performReservation: re-checa overlap, persiste reserva pending, atualiza sala para reserved, cria aprovação 1º nível
- **[] T-15.3** - Toast + redirect para /reservas

Recomendação de implementação - NAO marcar room.status='reserved' globalmente. Calcular status da sala dinamicamente a partir das reservas ativas em 'agora', preservando outras janelas livres.

Épico 8 - Minhas Reservas (CRUD)

US-16 · Como professor, quero ver todas as minhas reservas com filtros por status e busca textual, para que eu encontre rapidamente o que preciso

Feature: F-15 · Listagem de reservas pessoais com filtros e busca

Tasks técnicas

- ☐ T-16.1 - Render base em src/modules/reservations.js
- ☐ T-16.2 - Filter chips (Todas/Pendentes/Aprovadas/Recusadas) com estado local
- ☐ T-16.3 - Busca textual via filterByText em room, purpose, requester
- ☐ T-16.4 - Limpeza de badge ao entrar (marca read: true)

US-17 · Como professor, quero ver, editar e cancelar minhas reservas pendentes, para que possa corrigir erros antes da aprovação

Feature: F-16 · Edição e cancelamento de reservas pendentes

Tasks técnicas

- ☐ T-17.1 - Modal "Ver detalhes"
- ☐ T-17.2 - Modal "Editar" (horário + finalidade, status preservado)
- ☐ T-17.3 - Cancelamento com confirm() + filter imutável
- ☐ T-17.4 - Reservas rejected ganham botão "Recorrer" (apenas toast no momento)

US-18 · Como professor, quero exportar minhas reservas em CSV, para arquivamento ou cruzamento com planilhas

Feature: F-17 · Exportação de reservas para CSV

Tasks técnicas

- ☐ T-18.1 - exportCSV() com Blob + URL.createObjectURL

Épico 9 - Fluxo de Aprovações

US-19 · Como admin, quero ver todas as aprovações pendentes consolidadas, para que possa decidir uma a uma

Feature: F-18 · Fila consolidada de aprovações pendentes

Tasks técnicas

- ☐ T-19.1 - Render de cards em src/modules/approvals.js
- ☐ T-19.2 - Limpeza de badge ao entrar

US-20 · Como admin, quero aprovar/recusar com 1 clique e o solicitante ser notificado, para que o ciclo se complete

Feature: F-19 · Decisão de aprovação com notificação ao autor

Tasks técnicas

- ☐ T-20.1 - resolveApproval chama resolveUserApproval no path do solicitante
- ☐ T-20.2 - Fallback para seed sem requesterEmail
- ☐ T-20.3 - Toast + atualização de badges

Épico 10 - Gestão de Salas (Admin)

US-21 · Como admin, quero CRUD completo de salas com filtro por status, para gerir o inventário físico

Feature: F-20 · CRUD de salas com filtros e recursos

Tasks técnicas

- ☐ T-21.1 - Grid de cards
- ☐ T-21.2 - Modal create/edit com nome, tipo (Sala/Lab/Auditório), capacidade, bloco, recursos (checkboxes)
- ☐ T-21.3 - Filter chips: Todas/Disponíveis/Ocupadas/Em manutenção
- ☐ T-21.4 - Card "Adicionar" sempre ao final do grid
- ☐ T-21.5 - Modal de detalhes ao clicar no card
- ☐ T-21.6 - Exclusão com confirm()
- ☐ T-21.7 - Propagação para todos os usuários quando admin salva

Épico 11 - Gestão de Usuários (Admin)

US-22 · Como admin, quero CRUD de usuários com perfis professor/admin, para controlar quem usa o sistema

Feature: F-21 · CRUD de usuários

Tasks técnicas

- ☐ T-22.1 - Tabela com avatar, nome, e-mail, perfil
- ☐ T-22.2 - Modal create/edit
- ☐ T-22.3 - Busca por nome/e-mail/role
- ☐ T-22.4 - Remoção com confirm()

US-23 · Como admin, quero revisar solicitações de cadastro pendentes e aprovar/recusar, para controlar o onboarding

Feature: F-22 · Revisão de solicitações de cadastro pendentes

Tasks técnicas

- ☐ T-23.1 - Botão "Solicitações de Cadastro" abre modal
- ☐ T-23.2 - Aprovar - cria usuário + marca approved: true
- ☐ T-23.3 - Recusar - remove do array de signups
- ☐ T-23.4 - Reabre modal automaticamente se restarem pendências

O fluxo de signup já persiste em localStorage["sira:signups"] (US-04 done). Falta o consumidor admin.

Épico 12 - Notificações

US-24 · Como admin, quero ver minhas notificações em ordem cronológica e marcar como lidas, para acompanhar eventos do sistema

Feature: F-23 · Caixa de notificações com marcação como lida

Tasks técnicas

- **[] T-24.1** - Render de lista em src/modules/notifications.js
- **[] T-24.2** - Botão "Marcar todas como lidas"
- **[] T-24.3** - Atualização imutável via Array.map
- **[] T-24.4** - Sincroniza badge da sidebar após mutação

Épico 13 - Componentes Reutilizáveis

US-25 · Como dev, quero um sistema de modais centralizado com createModal/openModal/closeModal, para evitar duplicação visual

Feature: F-24 · Sistema de modais centralizado

Tasks técnicas

- **[] T-25.1** - src/components/modal.js com initModalListeners
- **[] T-25.2** - Fechamento por Escape (event delegation global)
- **[x] T-25.3** - toast() e confirm() reutilizáveis em src/utils/dom.js

toast() e confirm() já existem em utils/dom.js (US-02). Falta criar createModal/openModal/closeModal e o listener global de Escape.

7. Resumo quantitativo

Estado	Épicos	Features	User stories	Tasks
Mergeado em develop	8 completos + parte do 8º	15 (F-01 a F-14 + F-24)	15 (US-01 a US-15 + US-25)	≈ 49
Pendente (escopo restante)	4 completos + parte do 8º	9 (F-11, F-15-23)	10 (US-12, 16-24)	≈ 32
Total geral	13	24	25	≈ 81

Métricas de código (estado atual em main) - aproximadamente 1.980 linhas em 8 arquivos (4 JS + 3 CSS + 1 JSON): main.js (216), data/store.js (54), utils/dom.js (118), utils/fp.js (135), style.css (1.014), home.css (298), auth.css (145), data/logins.json (1). Ainda não existem src/modules/ nem src/components/.

Recomendação para a próxima sprint

Caminho crítico para destravar todas as features de produto: US-10/US-11 (camada de persistência por usuário) -> US-25 (modais) + US-06/US-07 (sidebar + roteamento) -> US-05/US-09 (logout + dark mode, dependem da sidebar). Sem isso, nenhuma das páginas de dashboard, calendário, reservas, aprovações, salas, usuários ou notificações pode renderizar dados reais.

Boas práticas a aplicar já na primeira implementação (evitam refator futuro): vincular aprovação a reserva por **reservationId determinístico** (em vez de heurística de string), calcular status de sala por **janela de tempo** (em vez de mutação global em room.status) e personalizar badges de não-lidas por requesterEmail === CURRENT_USER.email.

8. Tarefas individuais - Gabriel Marques

Gabriel Marques

Fundação + Autenticação + Tema

User stories	6
Tasks técnicas	20
Arquivos para enviar	12
Commits sugeridos	7
Distribuição	Membro 1 de 5 - 4 US done (US-01 a US-04), 2 pendentes (US-05, US-09)

User Stories sob responsabilidade

US-01 · Setup Vite vanilla

Feature: F-01 · Scaffold Vite + ESM com deploy para GitHub Pages

Critérios de aceitação

- `npm install && npm run dev` sobe servidor
- `npm run build` gera `dist/`
- Projeto usa ESM nativo

Tasks técnicas

- [x] T-01.1 - Inicializar `package.json` com scripts `dev/build/preview` e Vite `^5.2.0`
- [x] T-01.2 - Criar `vite.config.js` com `root: '.'` e `outDir: 'dist'`
- [x] T-01.3 - Criar `index.html` com `<script type="module" src="/src/main.js">`
- [x] T-01.4 - Configurar `.gitignore` para `node_modules/` e `dist/`
- [x] T-01.5 - Documentar setup no `README.md`

US-02 · Utilitários funcionais e CSS global

Feature: F-02 · Biblioteca de utilitários funcionais e design system base

Tasks técnicas

- [x] T-02.1 - `src/utlis/fp.js`: `filterByText`, `filterByStatus`, `filterRoomsByStatus`, `computeStats`, `initials`, `roleLabel`, `statusBadge`, `roomStatusInfo`
- [x] T-02.2 - `src/utlis/dom.js`: `el()`, `render()`, `btn()`, `badge()`, `tableRow()`, `toast()`, `confirm()`
- [x] T-02.3 - CSS global (`style.css`, `home.css`, `auth.css`) com CSS Variables e dark mode

US-03 · Login institucional

Feature: F-03 · Login institucional com sessão persistente

Critérios de aceitação

- Tela de login renderizada quando `CURRENT_USER === null`
- Login válido persiste sessão em `localStorage["sira-auth"]`
- Login inválido exibe alerta "Usuário não encontrado"

Tasks técnicas

- **[x] T-03.1** - Implementar `login()`, `logout()`, `tryRestoreSession()` em `src/data/store.js`
- **[x] T-03.2** - Criar tela de login inline em `src/main.js`
- **[x] T-03.3** - Bootstrapping com `tryRestoreSession()` antes de montar shell
- **[x] T-03.4** - Seed do admin em `src/data/logins.json`

US-04 · Solicitação de cadastro

Feature: F-04 · Auto-serviço de cadastro de professor

Critérios de aceitação

- Formulário com nome, e-mail, perfil (professor)
- Solicitação persiste em `localStorage["sira:signups"]`
- Após envio, mostra alerta de confirmação e volta ao login

Tasks técnicas

- **[x] T-04.1** - Implementar `renderSignup()` em `src/main.js`
- **[x] T-04.2** - Validação de campos obrigatórios
- **[x] T-04.3** - Geração de ID `su-<timestamp>`

US-05 · Logout (PENDENTE - depende da sidebar de Ian)

Feature: F-05 · Encerramento de sessão

Tasks técnicas

- **[] T-05.1** - Botão "Sair" no `userPill` da sidebar
- **[] T-05.2** - Limpeza de `localStorage["sira-auth"]` ao sair

US-09 · Tema claro/escuro (PENDENTE - depende da sidebar de Ian)

Feature: F-08 · Tema claro/escuro persistente

Tasks técnicas

- **[] T-09.1** - Toggle `dark-toggle` no rodapé da sidebar
- **[] T-09.2** - Persistência em `localStorage["sira-theme"]`
- **[] T-09.3** - Aplicação na `<html>.dark` no bootstrap

Arquivos para enviar ao GitHub

#	Arquivo	Tipo	Conteúdo da contribuição
1	<code>package.json</code>	DONE	Vite ^8.0.10 + scripts dev/build/preview/lint/format/prepare

#	Arquivo	Tipo	Conteúdo da contribuição
2	vite.config.js	DONE	Config Vite com base condicional (GITHUB_PAGES)
3	index.html	DONE	Entry point HTML com <script type="module" src="/src/main.js">
4	.gitignore	DONE	Ignora node_modules/, dist/, scripts auxiliares e ESLint cache
5	README.md	DONE	Documentação completa, roadmap da entrega, deploy
6	src/utils/fp.js	DONE	Utils funcionais: filterByText, computeStats, initials, roleLabel, statusBadge, roomStatusInfo
7	src/utils/dom.js	DONE	Helpers DOM: el(), render(), btn(), badge(), tableRow(), toast(), confirm()
8	src/style.css	DONE	Estilos globais + CSS Variables (1.014 linhas)
9	src/auth.css	DONE	Estilos de login/cadastro (145 linhas)
10	src/data/logins.json	DONE	Seed do usuário admin (admin@ifpb.edu.br)
11	src/data/store.js	PARCIAL	Bloco AUTH já existe (CURRENT_USER, login, logout, tryRestoreSession, getUsersGlobal). Falta o bloco PERSISTENCIA do Igor (US-10).
12	src/main.js	PARCIAL	BOOTSTRAP + tela de login + tela de signup já existem. FALTA: toggle dark mode (US-09) e listener de logout (US-05) - ambos dependem da sidebar.

Sequência sugerida de commits

Cada commit deve ser atômico (1 feature por vez) e seguir Conventional Commits.

Commit 1

```
git commit -m "chore: scaffold Vite vanilla project (DONE - merged in PR #125)"
```

Arquivos: package.json, vite.config.js, index.html, .gitignore, README.md

Commit 2

```
git commit -m "feat(utils): add fp.js with map/filter/reduce helpers (DONE - merged in PR #126)"
```

Arquivos: src/utils/fp.js

Commit 3

```
git commit -m "feat(utils): add dom.js with createElement factory and UI helpers (DONE - merged in PR #126)"
```

Arquivos: src/utils/dom.js

Commit 4

```
git commit -m "style: global CSS with variables and dark-mode auth styles (DONE - merged in PR #126)"
```

Arquivos: src/style.css, src/auth.css, src/home.css

Commit 5

```
git commit -m "feat(auth): login/logout/session restore via localStorage (DONE - merged in PR #127)"
```

Arquivos: src/data/store.js, src/data/logins.json

Commit 6

```
git commit -m "feat(auth): inline login + signup screens in bootstrap (DONE - merged in PR #127, #128)"
```

Arquivos: src/main.js

Commit 7

```
git commit -m "feat(theme): persistent dark-mode toggle (PENDENTE - depende da sidebar)"
```

Arquivos: src/main.js

Workflow Git recomendado

1. Crie sua branch a partir de main

```
git checkout -b feat/gabriel-fundação
```

2. Adicione e committe seguindo a sequência acima

```
git add <arquivos>
```

```
git commit -m "<mensagem>"
```

3. Empurre a branch e abra Pull Request

```
git push -u origin feat/gabriel-fundação
```

```
gh pr create --title "feat: Fundação + Autenticação + Tema" \
--body "Implementa US: US-01, US-02, US-03, US-04 (US-05, US-09 dependem da sidebar de Ian)"
```

Checklist antes do push

- ☒ Rodei npm install && npm run dev e a aplicação subiu sem erros
- ☒ Naveguei manualmente por todas as páginas que minha entrega afeta
- ☒ Conferi que npm run build gera dist/ sem warnings críticos
- ☒ Cada commit tem uma mensagem clara no padrão Conventional Commits
- ☒ Não incluo node_modules/ nem dist/ nos commits
- ☐ Todas as user stories da minha lista estão cobertas pelos commits
- ☒ Solicitei review de pelo menos 1 colega de outra área antes do merge

9. Tarefas individuais - Ian Lucas

Ian Lucas

Shell + Navegação + Modais + Calendário

User stories	5
Tasks técnicas	17
Arquivos para enviar	5
Commits sugeridos	5
Distribuição	Membro 2 de 5 - 0 / 17 tasks (todas pendentes - bloqueia US-05/US-09 do Gabriel)

User Stories sob responsabilidade

US-06 · Sidebar contextual com badges (PENDENTE)

Feature: F-06 · Navegação contextual por perfil (sidebar + URLs limpas)

Critérios de aceitação

- Sidebar com seções VISAO GERAL, RESERVAS, ADMINISTRACAO
- Itens visíveis variam por role
- Item ativo destacado e badges numéricos para não lidos

Tasks técnicas

- **[] T-06.1** - Componente src/components/sidebar.js com NAV_ITEMS declarativo
- **[] T-06.2** - Filtro por role: professor ve apenas reservas, calendário, novaReserva
- **[] T-06.3** - Badges dinamicos via funções badge: () => ... lendo store
- **[] T-06.4** - window.updateSidebarBadges para atualização imperativa após mutações

US-07 · URLs limpas e voltar/avancar (PENDENTE)

Feature: F-06 · Navegação contextual por perfil (sidebar + URLs limpas)

Tasks técnicas

- **[] T-07.1** - Roteador funcional PAGE_RENDERERS em src/main.js
- **[] T-07.2** - window.history.pushState em navigate()
- **[] T-07.3** - Listener de popstate em src/main.js
- **[] T-07.4** - Fallback para calendário quando rota desconhecida ou usuário não-admin tenta acessar página restrita

US-08 · Drawer mobile (PENDENTE)

Feature: F-07 · Adaptação responsiva mobile (drawer + tabelas em cards)

Tasks técnicas

- **[] T-08.1** - Hamburger injetado dinamicamente via MutationObserver
- **[] T-08.2** - Overlay clicavel que fecha o drawer
- **[] T-08.3** - addTableLabels() injeta data-label para tabelas responsivas

US-13 · Calendário semanal 7d x 12h (PENDENTE - depende de US-10/US-11 do Igor)

Feature: F-12 · Grade semanal de reservas (7d x 12h)

Critérios de aceitação

- Grade 7 dias x 12 horas (07:00 - 18:00)
- Eventos coloridos por status
- Navegação semana anterior / próxima

Tasks técnicas

- **[] T-13.1** - getWeekDates(offset) em src/modules/calendar.js
- **[] T-13.2** - reservationsToEvents (parsing data/hora + spread em blocos)
- **[] T-13.3** - Render da grade dinamica (rebuildCalendar)

US-25 · Sistema de modais centralizado (PARCIAL - toast/confirm já em utils/dom.js)

Feature: F-24 · Sistema de modais centralizado

Tasks técnicas

- **[] T-25.1** - src/components/modal.js com initModalListeners
- **[] T-25.2** - Fechamento por Escape (event delegation global)
- **[x] T-25.3** - toast() e confirm() reutilizáveis em src/utils/dom.js (DONE em US-02)

Arquivos para enviar ao GitHub

#	Arquivo	Tipo	Conteúdo da contribuição
1	src/components/sidebar.js	NOVO	Sidebar dinamica com NAV_ITEMS, badges, filtro por role, user pill, sair
2	src/components/modal.js	NOVO	Sistema centralizado: createModal, openModal, closeModal, initModalListeners (Escape)
3	src/modules/calendar.js	NOVO	Grade semanal 7d x 12h, getWeekDates, reservationsToEvents, navegação de semana
4	src/home.css	NOVO	Estilos do calendário + grid responsivo
5	src/main.js	PARCIAL	Bloco ROUTING: PAGE_RENDERERS, navigate(), pushState/popstate, restrição por role; Bloco MOBILE: openSidebar/closeSidebar, overlay, MutationObserver do hamburger, addTableLabels

Sequência sugerida de commits

Cada commit deve ser atômico (1 feature por vez) e seguir Conventional Commits.

Commit 1

```
git commit -m "feat(shell): sidebar with role-based nav and live badges"
```

Arquivos: src/components/sidebar.js

Commit 2

```
git commit -m "feat(modal): reusable modal system with Escape close"
```

Arquivos: src/components/modal.js

Commit 3

```
git commit -m "feat(routing): URL routing with pushState and popstate"
```

Arquivos: src/main.js

Commit 4

```
git commit -m "feat(mobile): hamburger drawer + overlay + responsive table labels"
```

Arquivos: src/main.js

Commit 5

```
git commit -m "feat(calendar): weekly grid view with status-colored events"
```

Arquivos: src/modules/calendar.js, src/home.css

Workflow Git recomendado

1. Crie sua branch a partir de main

```
git checkout -b feat/ian-shell
```

2. Adicione e commite seguindo a sequência acima

```
git add <arquivos>
```

```
git commit -m "<mensagem>"
```

3. Empurre a branch e abra Pull Request

```
git push -u origin feat/ian-shell
```

```
gh pr create --title "feat: Shell + Navegação + Modais + Calendário" \
--body "Implementa US: US-06, US-07, US-08, US-13, US-25"
```

Checklist antes do push

- **[]** Rodei npm install && npm run dev e a aplicação subiu sem erros
- **[]** Naveguei manualmente por todas as páginas que minha entrega afeta
- **[]** Conferi que npm run build gera dist/ sem warnings críticos
- **[]** Cada commit tem uma mensagem clara no padrão Conventional Commits
- **[]** Não incluo node_modules/ nem dist/ nos commits
- **[]** Todas as user stories da minha lista estão cobertas pelos commits
- **[]** Solicitei review de pelo menos 1 colega de outra área antes do merge

10. Tarefas individuais - Igor Gimenez

Igor Gimenez

Persistência + Dashboard + Notificações

User stories	4
Tasks técnicas	16
Arquivos para enviar	4
Commits sugeridos	4
Distribuição	Membro 3 de 5 - 1 / 16 tasks (US-10/US-11 destrava todas as outras features)

User Stories sob responsabilidade

US-10 · LocalStorage isolado por usuário (PENDENTE - prioridade ALTA)

Feature: F-09 · Dados isolados por usuário no LocalStorage

Critérios de aceitação

- Caminho de persistência: `sira_db/<email>/<coleção>.json`
- Sem usuário logado, `loadCollection` retorna apenas o seed
- Admin consegue consolidar dados de todos os usuários

Tasks técnicas

- **[] T-10.1** - `loadCollection/saveCollection` por path em `src/data/store.js`
- **[] T-10.2** - Getters: `getRooms`, `getReservations`, `getUsers`, `getNotifications`, `getApprovals`
- **[] T-10.3** - Consolidação admin em `getReservations` e `getApprovals`
- **[] T-10.4** - `saveRooms/saveUsers` propagando para todos quando admin salva
- **[] T-10.5** - `genId(prefix)` para chaves únicas
- **[] T-10.6** - Seeds vazios em `src/data/seed.json`

US-11 · Sincronização aprovação -> reserva -> notificação (PENDENTE)

Feature: F-10 · Sincronização aprovação → reserva → notificação

Tasks técnicas

- **[] T-11.1** - `resolveUserApproval(approvalId, email, decision)` em `src/data/store.js`
- **[] T-11.2** - Remove aprovação, atualiza status da reserva e cria notificação no diretório do usuário-alvo

US-12 · Dashboard com KPIs em tempo real (PARCIAL)

Feature: F-11 · Painel administrativo de KPIs em tempo real

Tasks técnicas

- ☒ T-12.1 - computeStats com Array.reduce em src/utils/fp.js (DONE em US-02)
- ☐ T-12.2 - Render de 4 stat-cards via Array.map em src/modules/dashboard.js
- ☐ T-12.3 - Tabela "Reservas aprovadas recentes" (slice 3)
- ☐ T-12.4 - Placeholder de gráfico de ocupação

US-24 · Caixa de notificações (PENDENTE)

Feature: F-23 · Caixa de notificações com marcação como lida

Tasks técnicas

- ☐ T-24.1 - Render de lista em src/modules/notifications.js
- ☐ T-24.2 - Botão "Marcar todas como lidas"
- ☐ T-24.3 - Atualização imutável via Array.map
- ☐ T-24.4 - Sincroniza badge da sidebar após mutação

Arquivos para enviar ao GitHub

#	Arquivo	Tipo	Conteúdo da contribuição
1	src/data/seed.json	NOVO	Seed vazio para rooms / reservations / notifications / approvals
2	src/data/store.js	PARCIAL	Bloco PERSISTENCIA: loadCollection/saveCollection por path, getters com consolidação admin, setters com propagação admin, resolveUserApproval, genId
3	src/modules/dashboard.js	NOVO	Stat cards via Array.reduce, tabela de reservas recentes, placeholder de gráfico
4	src/modules/notifications.js	NOVO	Lista, marcar individual / em massa, sync com badge da sidebar

Sequência sugerida de commits

Cada commit deve ser atômico (1 feature por vez) e seguir Conventional Commits.

Commit 1

```
git commit -m "feat(store): per-user localStorage persistence with admin consolidation"
```

Arquivos: src/data/store.js, src/data/seed.json

Commit 2

```
git commit -m "feat(store): resolveUserApproval cross-user side effects"
```

Arquivos: src/data/store.js

Commit 3

```
git commit -m "feat/dashboard): real-time stats with Array.reduce"
```

Arquivos: src/modules/dashboard.js

Commit 4

```
git commit -m "feat(notifications): list and mark-as-read flow"
```

Arquivos: src/modules/notifications.js

Workflow Git recomendado

1. Crie sua branch a partir de main

```
git checkout -b feat/igor-persistência
```

2. Adicione e commite seguindo a sequência acima

```
git add <arquivos>
```

```
git commit -m "<mensagem>"
```

3. Empurre a branch e abra Pull Request

```
git push -u origin feat/igor-persistência
```

```
gh pr create --title "feat: Persistência + Dashboard + Notificações" \
--body "Implementa US: US-10, US-11, US-12, US-24"
```

Checklist antes do push

- ☐ Rodei npm install && npm run dev e a aplicação subiu sem erros
- ☐ Naveguei manualmente por todas as páginas que minha entrega afeta
- ☐ Conferi que npm run build gera dist/ sem warnings críticos
- ☐ Cada commit tem uma mensagem clara no padrão Conventional Commits
- ☐ Não incluo node_modules/ nem dist/ nos commits
- ☐ Todas as user stories da minha lista estão cobertas pelos commits
- ☐ Solicitei review de pelo menos 1 colega de outra área antes do merge

11. Tarefas individuais - José Henrique

José Henrique

Reservas (Nova Reserva + CRUD + CSV)

User stories	5
Tasks técnicas	16
Arquivos para enviar	2
Commits sugeridos	5
Distribuição	Membro 4 de 5 - 0 / 16 tasks (depende de Igor + Ian + componentes/modal)

User Stories sob responsabilidade

US-14 · Buscar salas com filtros + anti-conflito (PENDENTE)

Feature: F-13 · Busca anti-conflito de salas

Critérios de aceitação

- Filtro por tipo (Sala / Laboratório / Auditório)
- Filtro por sobreposição de horário (exclui salas com reserva no intervalo, exceto rejected)
- Validação timeStart < timeEnd
- Suporte a recorrência semanal (chips por dia)

Tasks técnicas

- **[] T-14.1** - Formulário em src/modules/novaReserva.js
- **[] T-14.2** - parseTimeStr para tempos com 'h' ou ':'
- **[] T-14.3** - searchRooms com filtro de overlap
- **[] T-14.4** - Botões dia-da-semana com toggle visual

US-15 · Reserva 1-clique a partir do detalhe (PENDENTE)

Feature: F-14 · Reserva express com 1 clique

Tasks técnicas

- **[] T-15.1** - showRoomDetailsModal
- **[] T-15.2** - performReservation: gera reservationId, cria aprovação com reservationId determinístico, NAO mutar room.status global
- **[] T-15.3** - Toast + redirect para /reservas

US-16 · Listar reservas com filtros + busca (PENDENTE)

Feature: F-15 · Listagem de reservas pessoais com filtros e busca

Tasks técnicas

- ☐ T-16.1 - Render base em src/modules/reservations.js
- ☐ T-16.2 - Filter chips (Todas / Pendentes / Aprovadas / Recusadas)
- ☐ T-16.3 - Busca textual via filterByText em room, purpose, requester
- ☐ T-16.4 - Limpeza de badge ao entrar (marca read: true)

US-17 · Ver / editar / cancelar reservas (PENDENTE)

Feature: F-16 · Edição e cancelamento de reservas pendentes

Tasks técnicas

- ☐ T-17.1 - Modal "Ver detalhes"
- ☐ T-17.2 - Modal "Editar" (horário + finalidade, status preservado)
- ☐ T-17.3 - Cancelamento com confirm() + filter imutável
- ☐ T-17.4 - Reservas rejected ganham botão "Recorrer"

US-18 · Exportar reservas em CSV (PENDENTE)

Feature: F-17 · Exportação de reservas para CSV

Tasks técnicas

- ☐ T-18.1 - exportCSV() com Blob + URL.createObjectURL

Arquivos para enviar ao GitHub

#	Arquivo	Tipo	Conteúdo da contribuição
1	src/modules/novaReserva.js	NOVO	Formulario completo de nova reserva: filtros (data, hora, tipo, finalidade), recorrência semanal, parseTimeStr, searchRooms com filtro de overlap, showRoomDetailsModal, performReservation
2	src/modules/reservations.js	NOVO	CRUD completo: listagem com filter chips, busca textual, modais de Ver / Editar / Cancelar, botão Recorrer para rejected, exportCSV via Blob

Sequência sugerida de commits

Cada commit deve ser atômico (1 feature por vez) e seguir Conventional Commits.

Commit 1

```
git commit -m "feat(nova-reserva): search form with overlap filtering and weekly recurrence"
```

Arquivos: src/modules/novaReserva.js

Commit 2

```
git commit -m "feat(nova-reserva): reservation creation with pending approval workflow"
```

Arquivos: src/modules/novaReserva.js

Commit 3

```
git commit -m "feat(reservations): CRUD with status filters and text search"
```

Arquivos: src/modules/reservations.js

Commit 4

```
git commit -m "feat(reservations): view, edit and cancel modals"
```

Arquivos: src/modules/reservations.js

Commit 5

```
git commit -m "feat(reservations): CSV export via Blob URL"
```

Arquivos: src/modules/reservations.js

Workflow Git recomendado

1. Crie sua branch a partir de main

```
git checkout -b feat/josé-reservas
```

2. Adicione e commite seguindo a sequência acima

```
git add <arquivos>
```

```
git commit -m "<mensagem>"
```

3. Empurre a branch e abra Pull Request

```
git push -u origin feat/josé-reservas
```

```
gh pr create --title "feat: Reservas (Nova Reserva + CRUD + CSV)" \
--body "Implementa US: US-14, US-15, US-16, US-17, US-18"
```

Checklist antes do push

- ☐ Rodei npm install && npm run dev e a aplicação subiu sem erros
- ☐ Naveguei manualmente por todas as páginas que minha entrega afeta
- ☐ Conferi que npm run build gera dist/ sem warnings críticos
- ☐ Cada commit tem uma mensagem clara no padrão Conventional Commits
- ☐ Não incluo node_modules/ nem dist/ nos commits
- ☐ Todas as user stories da minha lista estão cobertas pelos commits
- ☐ Solicitei review de pelo menos 1 colega de outra área antes do merge

12. Tarefas individuais - Pedro Sales

Pedro Sales

Admin (Aprovações + Salas + Usuários)

User stories	5
Tasks técnicas	20
Arquivos para enviar	3
Commits sugeridos	4
Distribuição	Membro 5 de 5 - 0 / 20 tasks (consume tudo: persistência, modais, sidebar)

User Stories sob responsabilidade

US-19 · Fila de aprovações pendentes (PENDENTE)

Feature: F-18 · Fila consolidada de aprovações pendentes

Tasks técnicas

- ☐ T-19.1 - Render de cards em src/modules/approvals.js
- ☐ T-19.2 - Limpeza de badge ao entrar

US-20 · Aprovar / recusar com 1 clique e notificar solicitante (PENDENTE)

Feature: F-19 · Decisão de aprovação com notificação ao autor

Tasks técnicas

- ☐ T-20.1 - resolveApproval chama resolveUserApproval no path do solicitante
- ☐ T-20.2 - Fallback para seed sem requesterEmail
- ☐ T-20.3 - Toast + atualização de badges

US-21 · CRUD de salas com filtros e recursos (PENDENTE)

Feature: F-20 · CRUD de salas com filtros e recursos

Tasks técnicas

- ☐ T-21.1 - Grid de cards
- ☐ T-21.2 - Modal create/edit (nome, tipo, capacidade, bloco, recursos)
- ☐ T-21.3 - Filter chips: Todas / Disponíveis / Ocupadas / Em manutenção
- ☐ T-21.4 - Card "Adicionar" sempre ao final do grid
- ☐ T-21.5 - Modal de detalhes ao clicar no card
- ☐ T-21.6 - Exclusão com confirm()
- ☐ T-21.7 - Propagação para todos os usuários quando admin salva

US-22 · CRUD de usuários com perfis (PENDENTE)

Feature: F-21 · CRUD de usuários

Tasks técnicas

- **[] T-22.1** - Tabela com avatar, nome, e-mail, perfil
- **[] T-22.2** - Modal create/edit
- **[] T-22.3** - Busca por nome/e-mail/role
- **[] T-22.4** - Remoção com confirm()

US-23 · Aprovação de cadastros pendentes (PENDENTE - dados já em sira:signups por US-04)

Feature: F-22 · Revisão de solicitações de cadastro pendentes

Tasks técnicas

- **[] T-23.1** - Botão "Solicitações de Cadastro" abre modal
- **[] T-23.2** - Aprovar - cria usuário + marca approved: true
- **[] T-23.3** - Recusar - remove do array de signups
- **[] T-23.4** - Reabre modal automaticamente se restarem pendências

Arquivos para enviar ao GitHub

#	Arquivo	Tipo	Conteúdo da contribuição
1	src/modules/approvals.js	NOVO	Lista de aprovações pendentes consolidadas, cards com Aprovar/Recusar, integração com resolveUserApproval, fallback para seed antigo
2	src/modules/rooms.js	NOVO	CRUD de salas: grid de cards, modal create/edit (nome/tipo/capacidade/bloco/recursos), filter chips por status, card 'Adicionar', modal de detalhes, exclusão
3	src/modules/users.js	NOVO	CRUD de usuários: tabela com avatar/nome/email/perfil, modal create/edit, busca, remoção + Modal de Solicitações de Cadastro pendentes

Sequência sugerida de commits

Cada commit deve ser atômico (1 feature por vez) e seguir Conventional Commits.

Commit 1

```
git commit -m "feat(approvals): admin approval flow with cross-user side effects"
```

Arquivos: src/modules/approvals.js

Commit 2

```
git commit -m "feat(rooms): full CRUD with status filters and resource management"
```

Arquivos: src/modules/rooms.js

Commit 3

```
git commit -m "feat(users): user CRUD with role-based profile selection"
```

Arquivos: src/modules/users.js

Commit 4

```
git commit -m "feat(users): signup request review modal"
```

Arquivos: src/modules/users.js

Workflow Git recomendado

1. Crie sua branch a partir de main

```
git checkout -b feat/pedro-admin
```

2. Adicione e commite seguindo a sequência acima

```
git add <arquivos>
```

```
git commit -m "<mensagem>"
```

3. Empurre a branch e abra Pull Request

```
git push -u origin feat/pedro-admin
```

```
gh pr create --title "feat: Admin (Aprovações + Salas + Usuários)" \
--body "Implementa US: US-19, US-20, US-21, US-22, US-23"
```

Checklist antes do push

- ☐ Rodei npm install && npm run dev e a aplicação subiu sem erros
- ☐ Naveguei manualmente por todas as páginas que minha entrega afeta
- ☐ Conferi que npm run build gera dist/ sem warnings críticos
- ☐ Cada commit tem uma mensagem clara no padrão Conventional Commits
- ☐ Não incluo node_modules/ nem dist/ nos commits
- ☐ Todas as user stories da minha lista estão cobertas pelos commits
- ☐ Solicitei review de pelo menos 1 colega de outra área antes do merge

13. Workflow Git e checklist final do release

Resumo do fluxo recomendado para a equipe inteira durante a integração dos 5 ramos de feature na main.

Etapa	Comando / ação	Quem	Estado
1	Cada membro cria sua branch a partir de main e commita seguindo a sequência da sua seção	Cada dev	Em andamento (Gabriel done; demais a fazer)
2	Push da branch e abertura de Pull Request	Cada dev	PRs #125-#128 mergeados; restante pendente
3	Code review cruzado (1 dev de outra área)	Equipe	Recorrente
4	Merge para main respeitando ordem de dependência: Igor (US-10/11) -> Ian (US-25/06/07/08) -> Gabriel (US-05/09) -> Ian (US-13) -> José -> Pedro -> Igor (US-12/24)	Equipe	Pendente
5	Tag de release v1.0.0 + criação da Release no GitHub	Mantenedor	Pendente
6	Workflow .github/workflows/deploy.yml publica em GitHub Pages automaticamente	GitHub Actions	Configurado (release-driven)

Checklist final do release

- ☐ Todos os PRs dos 5 ramos foram aprovados e mergeados na main (4 / 5+ PRs até agora)
- ☐ Build de produção gera dist/ sem warnings (`GITHUB\_PAGES=true npm run build`)
- ☐ Smoke test manual: login admin -> dashboard / salas / usuários / aprovações / notificações
- ☐ Smoke test manual: login professor -> calendário / nova reserva / minhas reservas
- ☐ Tema escuro persiste após refresh
- ☐ Drawer mobile abre / fecha em < 400px